

PD 分离

1 吞吐量 (Throughput) vs 有效吞吐量 (Goodput)

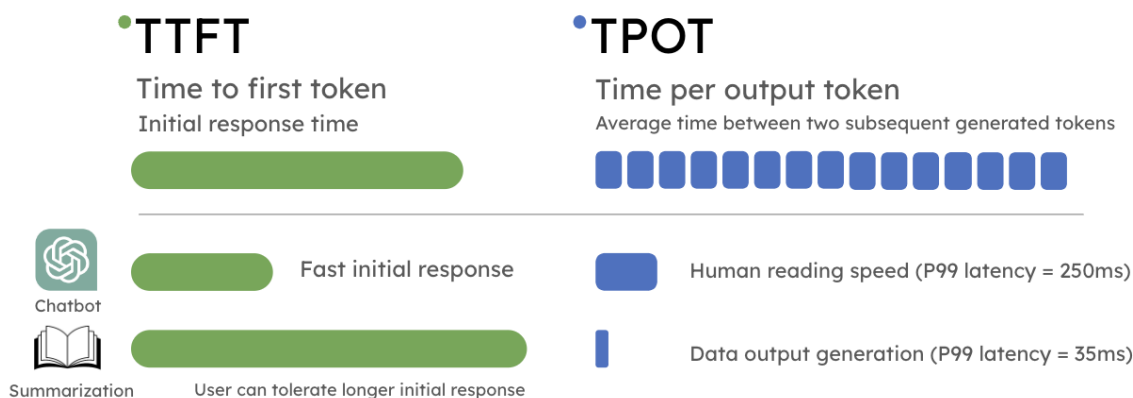
目前，大多数 LLM 服务系统（如 vLLM、TensorRT-LLM）都以吞吐量 (Throughput) 作为主要性能指标——即单位时间内处理的请求数 (RPS) 或生成的 token 数。这种度量方式直观，并且与成本 (\$/req) 有直接关联，因此被广泛采用。

实际上，下游应用的类型多种多样，它们在用户体验上的延迟需求各异，因此需要满足的服务等级目标 (SLO) 也存在显著差异。大模型服务中最常用的 SLO 包括：

- TTFT (Time To First Token): 首 token 响应延迟，直接影响用户的等待体验。
- TPOT (Time Per Output Token): 衡量两个连续生成的 token 之间的平均延迟，决定交互的流畅程度。

例如，实时聊天机器人更关注低 TTFT 以保证响应及时，而 TPOT 只需快于人类阅读速度（约 250 词/分钟）即可；相反，文档摘要则更强调低 TPOT，以便更快地产生完整摘要。

Applications have Diverse SLO



单纯依赖 Throughput 作为指标，并不能反映延迟表现，系统看似处理了大量请求，但其中不少未能满足 SLO，最终呈现给用户的仍是不理想的服务体验。

- **Throughput (吞吐量)**：通常指系统单位时间内处理的 token 数或请求数。很多工作把“提高吞吐量”作为主要优化目标，但在实际场景下，这并不直接代表用户体验。
- **Goodput (有效吞吐量)**：指系统在满足延迟约束（如 TTFT/TPOT SLO）的前提下，真正完成的请求数量。如果一个请求因为延迟过长而被用户放弃，或者超过服务约束而无效，那么即便它产生了 token，也不能算作有效产出。

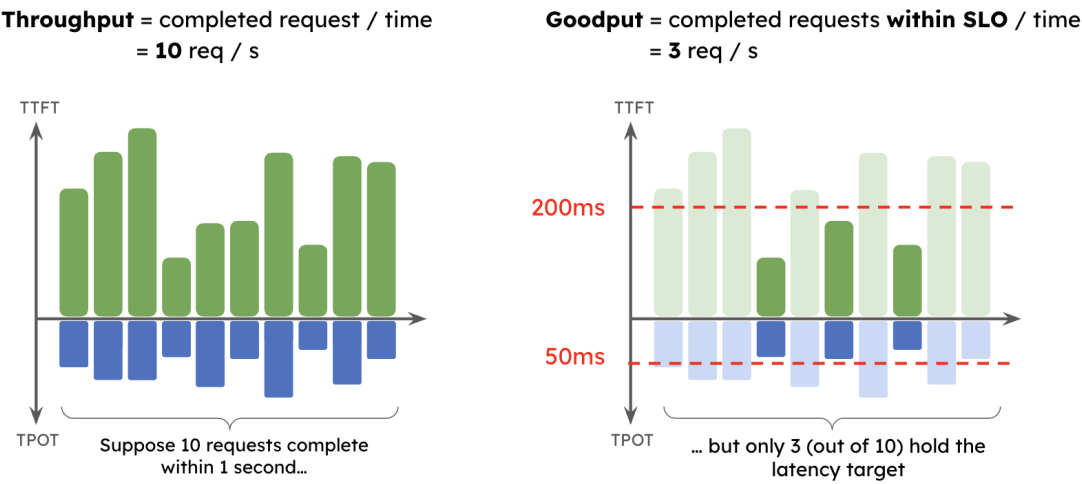
在 [DistServe](#) 论文中，引入了 Goodput 概念，**即在满足 SLO (TTFT 和 TPOT 要求) 的前提下，每秒完成的有效请求数**。与单纯的吞吐量相比，Goodput 是更优的衡量指标，因为它能够体现请求在满足 SLO 情况下的吞吐水平，从而同时反映成本效益与服务质量。

为了简要说明 Goodput，假设某个应用要求至少 90% 的请求满足 $TTFT < 200ms$ 且 $TPOT < 50ms$ ，则可以得到如下定义：

Goodput ($P_{90} TTFT < 200ms$ 且 $P_{90} TPOT < 50ms$) 表示在至少 90% 的请求同时满足 $TTFT < 200ms$ 和 $TPOT < 50ms$ 的条件下，系统所能维持的最大每秒请求数。

下图展示了一个简单的例子：某应用的吞吐量为 10 RPS（每秒请求数），但由于延迟约束的限制，只有 3 RPS 的请求满足 SLO，因此该系统的 Goodput 仅为 3 RPS。可以想象，用户在这样一个 高吞吐但低 Goodput 的系统中，依然会感受到较差的服务体验。

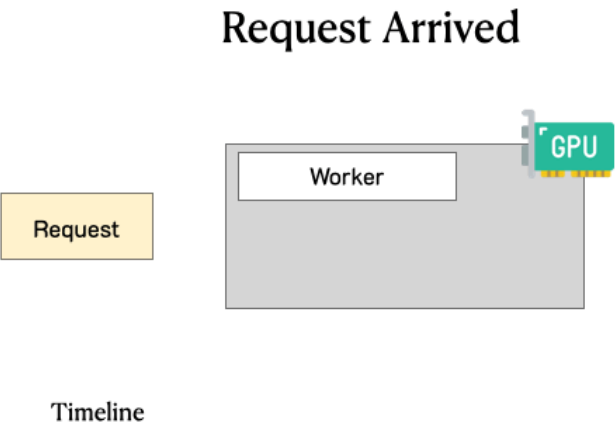
High Throughput ≠ High Goodput



2 Prefill 与 Decode 共置导致干扰

在 LLM 服务中请求的生命周期通常包含两个阶段：prefill（生成首个 token）和 decode（逐步生成后续 token）。大多数现有系统（如 vLLM、TensorRT-LLM）采用 continuous batching 技术，将 prefill 和 decode 混合在一起统一批处理。这种方式确实能够提升整体吞吐量，但由于两者计算特性和 SLO 目标差异显著，将它们共置在同一 GPU 上往往并不理想。

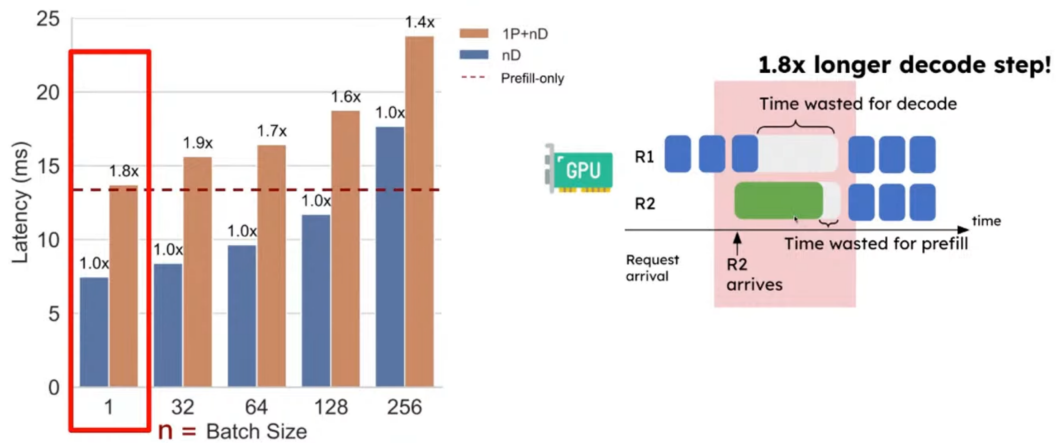
Disaggregation is a technique that



如下图所示，continuous batching 会带来明显的干扰。当 prefill 和 decode 被放在同一批次时，decode 请求的延迟（TPOT）会被显著拉长，而 prefill 请求的首 token 延迟（TTFT）也会有所增加。

Chunking P&D Cause Interference

Problem With Hybrid Batching (1P + nD)



(a) LLAMA-13B: 1P + nD (Prompt length = 128, 1xA100)

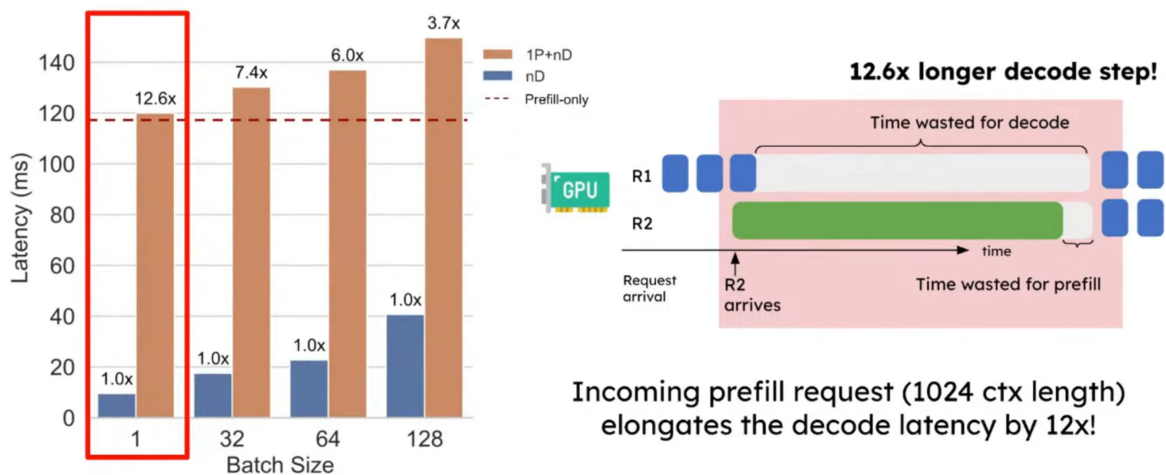
图中展示了三种不同的执行方式：

- 1P+nD (棕色柱子)：1 个 prefill 与 n 个 decode 混合批处理。
- nD (蓝色柱子)：仅包含 decode 请求的批处理。
- prefill-only (红色虚线)：仅运行 prefill 请求的延迟。

在 prompt 长度为 128 时，相比仅包含 decode 的请求，延迟增加约 1.8 倍；而当 prompt 长度为 1024 时，干扰效应显著放大，decode 延迟提升至 12.6 倍。

Chunking P&D Cause Interference

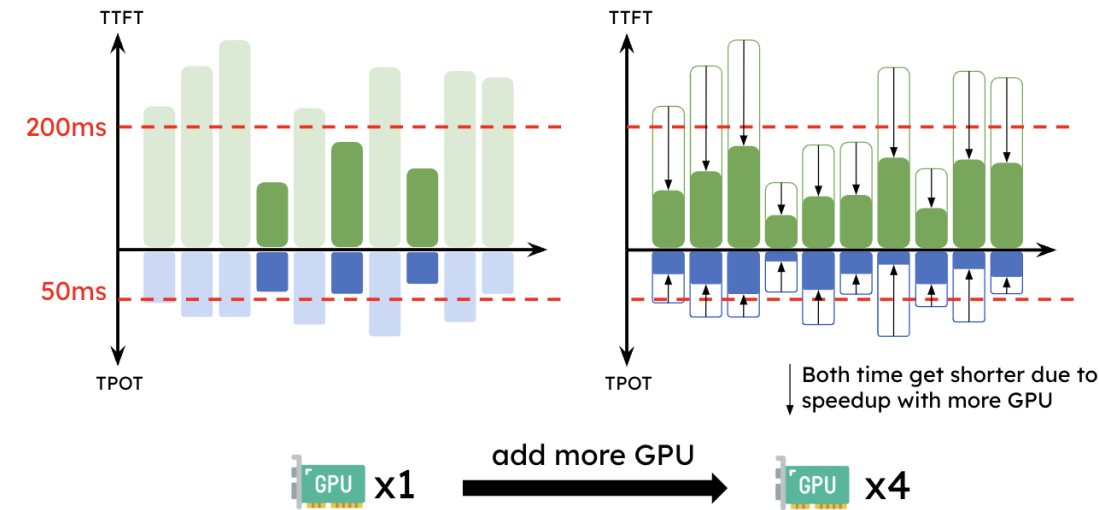
Problem With Hybrid Batching (1P + nD)



(b) LLAMA-13B: 1P + nD (Prompt length = 1024, 1xA100)

由于这种干扰，如下图所示，当服务必须同时满足 TTFT 和 TPOT 的 SLO 时，系统往往需要进行资源的过度配置才能达到延迟目标，尤其是在任一 SLO 要求较严格的情况下。

Colocation → Overprovision Resource to meet SLO



Cost	😎	😭
Goodput	😭	😎

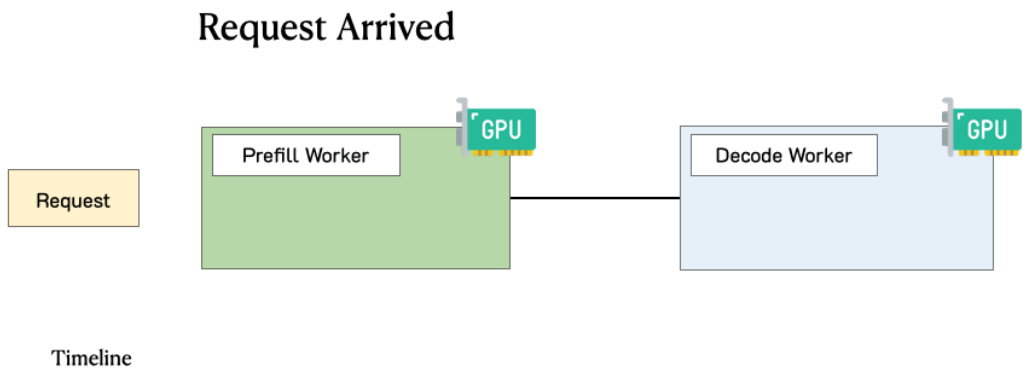
3 PD 分离的整体思路

直观的思路很简单：将 prefill 和 decode 分离到不同的 GPU 上，并为每个阶段定制并行策略。这自然解决了前面提到的两个问题：

- **没有干扰**：prefill 和 decode 各自独立运行，更快地完成计算，也更容易满足各自的 SLO。
- **资源分配与并行策略解耦**：可以针对 prefill 和 decode 分别进行优化。

下图展示了在这样一个分离式系统中，请求是如何被处理的。当一个请求到达系统时，它会先被分配到 prefill worker 完成 prefill 阶段；随后系统将其中间状态（主要是 KV Cache）迁移到 decode worker，并执行多步 decode 以生成后续 token；当生成完成后，请求才会离开系统。

Disaggregation is a technique that



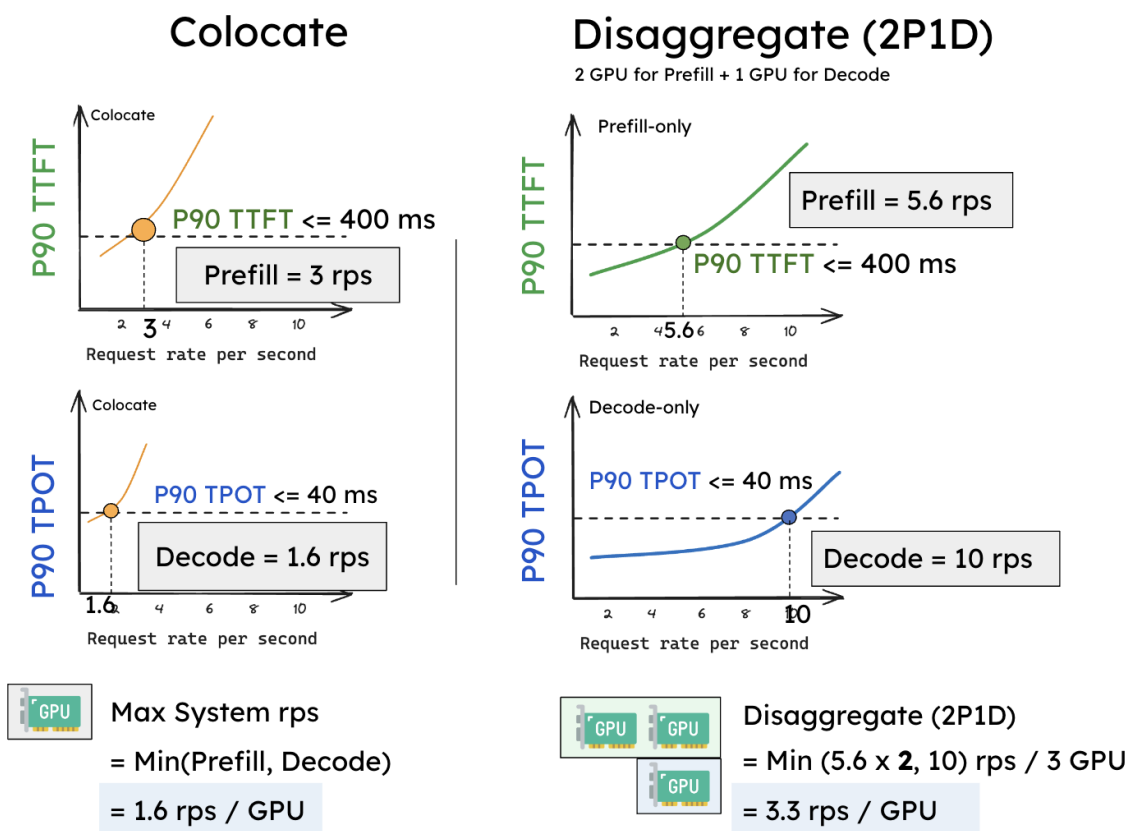
让我们通过一个简单的实验来看看为什么 PD 分离是有益的。我们在一张 A100-80GB GPU 上运行一个 130 亿参数的 LLM，请求到达服从泊松分布，输入长度为 512，输出长度为 64。我们逐步增加请求速率（x 轴），并在下图测量两类延迟（P90 TTFT 和 P90 TPOT，y 轴）的变化。

假设我们设定 SLO：P90 TTFT = 0.4 秒，P90 TPOT = 0.04 秒（下图中的横线）。实验结果表明：在单卡情况下，现有系统大约可以在 3 rps 下满足 TTFT 的延迟约束，而 TPOT 只能维持在 1.6 rps（下图左边）。由于必须同时满足两个约束条件，现有共置系统的 Goodput = $\min(3, 1.6) = 1.6$ rps/GPU。

在分离之后，性能得到了显著提升。如果单独处理一个阶段，prefill worker 和 decode worker 的 rps 都优于之前的结果——如下图右边所示，一个 prefill worker 大约可达到 5.6 rps，一个 decode worker 大约可达到 10 rps。更重要的是，我们现在可以灵活地分配资源，例如配置 2 个 prefill worker + 1 个 decode worker（记作 2P1D），共 3 张 GPU。此时：

$$\text{Goodput (2P1D)} = \min(5.6 \times 2, 10) = 10 \text{ reqs/s} \div 3 \text{ GPUs} \approx 3.3 \text{ rps/GPU}。$$

这个实验表明，即便没有引入任何并行优化，仅仅通过简单的分离，Goodput 就提升了约 2 倍。



4 分离式推理架构的优化方向

4.1 算力与存储

- prefill 阶段：拥有计算受限的性质（compute-bound），特别是在请求流量较大，用户的 prompt 也比较长的情况下。prefill 阶段算完 KV cache 并发给 decode 阶段后，理论上 prefill 就不再需要这个 KV cache 了（当然你也可以采用 LRU 等策略对 KV cache 的保存做管理，而不是一股脑地清除）。
- decode 阶段：拥有存储受限的性质（memory-bound），因为逐个 token 的生成方式，decode 阶段要频繁从存储中读取 KV Cache，同时也意味着它需要尽可能保存 KV cache。

因此在分离式框架下，计算和存储可以朝着两个独立的方向做优化。

4.2 Batching 策略

- prefill 阶段：随着 batch size 的增加，吞吐量的提升很快趋于平缓。这是因为 prefill 属于 compute-bound，当 batch 中的总 tokens 数超过一定规模后，GPU 的计算能力已经被完全吃满，再增加请求只会延长整体处理时间，而不会带来明显的吞吐提升。
- decode 阶段：随着 batch size 的增加，吞吐量的增长趋势越来越显著。这是因为 decode 阶段是 memory-bound，即相比于计算，读写数据的时间要更多。所以在 decode 阶段中，如果我们能提升 batch size，就能把计算强度提起来，吞吐量就上去了。

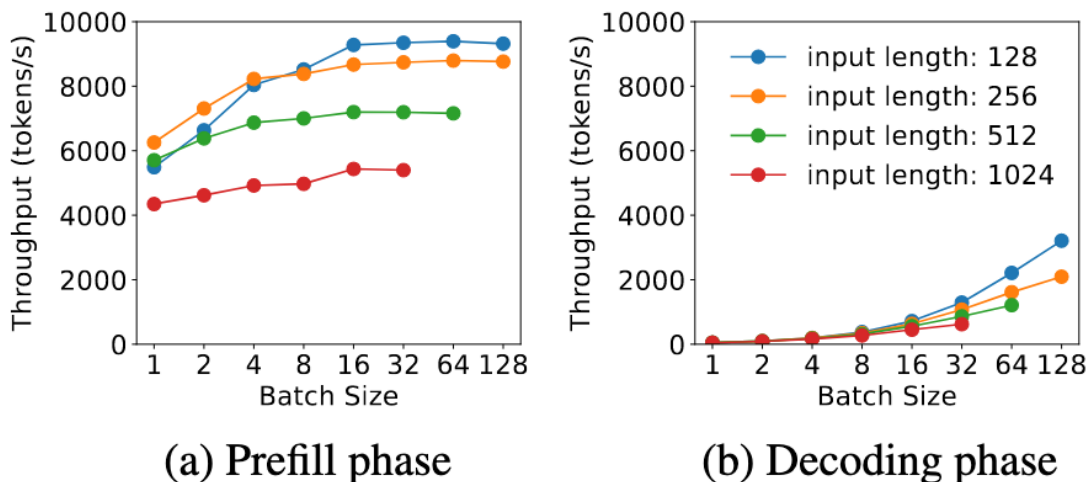


Figure 3: Throughput for two phases with different batch sizes and input lengths when serving an LLM with 13B parameters.

在分离架构下，我们可以针对 prefill 和 decode 的特性对 batching 策略分别进行优化：

- 具体来说，对于 prefill 实例，需要事先结合特定的 LLM 和 GPU 做性能分析，找出输入长度的临界点——一旦超过这个点，prefill 就会进入 compute-bound，此时增加 batch size 只会拖慢整体处理速度。在实际应用中，用户的 prompt 往往已有数百个 tokens，因此 **prefill 的 batch size 通常保持较小**。
- 相对地，decode 阶段更适合采用较大的 batch size，以充分提升 GPU 利用率和整体吞吐。

4.3 并行策略

由于 prefill 和 decode 具有不同的计算模式和延迟目标，这两个阶段的最佳并行策略通常并不相同。例如，当 TTFT 要求严格而 TPOT 要求相对宽松时，prefill 更适合采用**张量并行**来满足低延迟，而 decode 则通常采用**数据并行**或**流水线并行**来提升吞吐。

5 KV Cache 传输

PD 分离带来的代价是需要在 prefill 和 decode 的 GPU 之间传输中间状态（即 KV cache）。接下来，我们来看看 KV cache 传输的开销分析、传输方式以及相关的优化策略。

5.1 KV Cache 传输开销

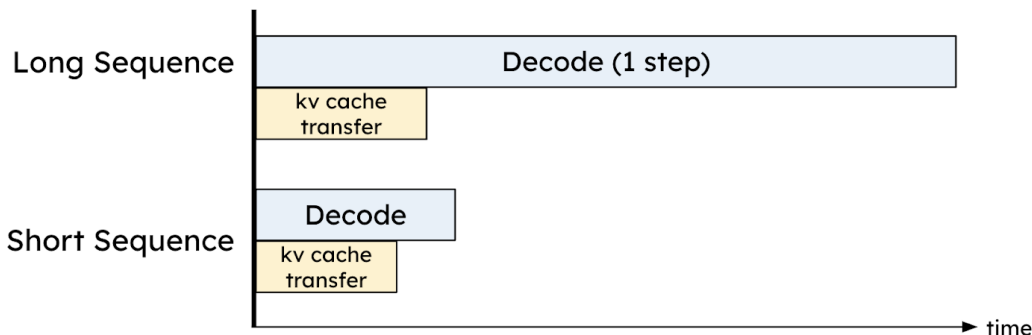
初看之下，KV cache 是 LLM 推理中巨大的内存开销，而 GPU 之间 KV cache 的传输似乎会成为瓶颈。然而，DistServe 的论文中展示了相反的结果：通过合理的放置，KV Cache 的传输开销可以被有效地最小化，甚至低于一次 decode 步骤的时间，这得益于当今高速互连网络（如 NVLink 和 PCI-e 5.0）。

假设我们在 GPU 之间使用 8 通道 PCIe 5.0 x16（每条链路 64GB/s）作为节点内互联。对于一个包含 2048 tokens 的请求，在服务 OPT-175B 时传输 KV cache 的延迟可以估算如下：

$$\text{Latency} = 2048 \text{ tokens} * (4.5 \text{ MB/token}) / (64\text{GB/s} * 8) = 17.6 \text{ ms}$$

对于 OPT-175B，延迟小于单次 decode 步骤（在 A100 上约为 30-50 毫秒）。对于更大的模型、更长的序列或更先进的网络（例如带宽为 600GB/s 的 A100-NVLink），如下图所示，与单次 decode 步骤相比，KV cache 传输相关的相对开销变得不那么显著。总之，通过精心安排 prefill 和 decode 工作节点以利用高带宽网络，可以有效隐藏 KV cache 传输的开销。

KV cache transfer is not the bottleneck



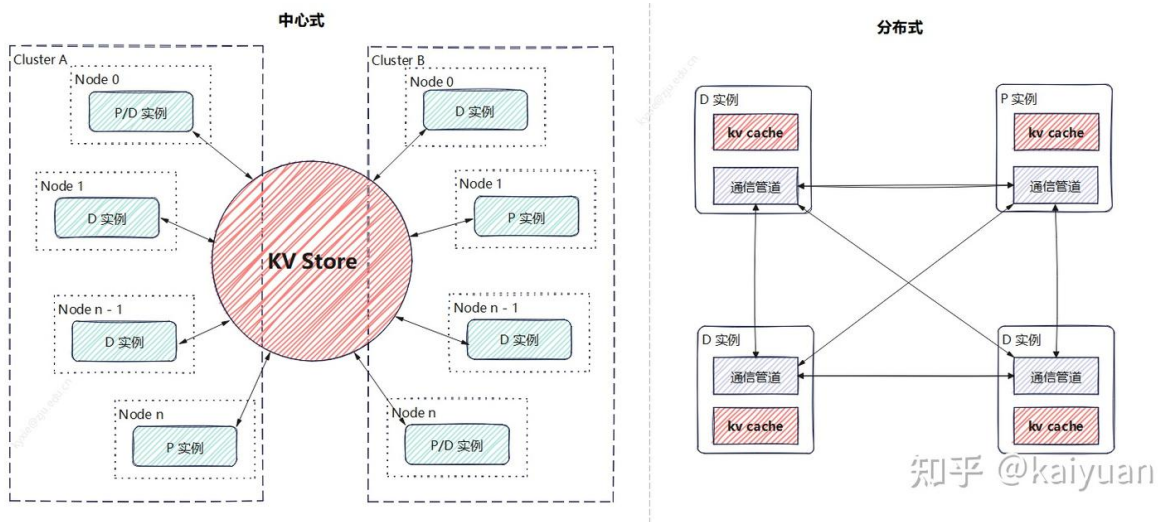
5.2 KV Cache 传输方式

目前 KV cache 的传输主要有两种方式：**中心存储**和**点对点 (P2P)**，当然在实际系统中也可能采用二者结合的混合方案。

- **中心存储**：建立一个跨设备的 KV store，由它统一管理 KV cache 的增、删、查和传递等操作。prefill 和 decode 实例只需与这个 KV store 交互，负责写入或读取数据。
- **P2P 传输**：每个实例独立管理自己的存储。例如，一个 prefill 实例完成计算后，会直接与目标 decode 实例建立通信，将 KV cache 传过去，不依赖统一的中介。

两种方式各有优劣：

- **中心存储**：更适合构建大规模集群，能充分利用多种存储介质和传输通道，并提升计算结果的复用效率，但在某些场景下性能可能受限，同时系统维护成本较高。
- **P2P 传输**：架构更简单，性能表现通常更好，但在扩展性和链路稳定性方面会面临挑战。



5.3 KV Cache 传输的网络堆栈

现有的物理数据链路可以分为 3 类：

- **Direct**，即 GPU 之间通过高速直连链路（如 NVLink 或 HCCS）相互连接。在这种情况下，可以利用底层的内存拷贝原语或集体通信库来完成数据传输。
- **Direct-NIC**，即 GPU 通过其配套的网卡（NIC）进行通信。在这里，可以使用定制化的库，通过 PCIe 和以太网（或 InfiniBand）进行数据传输。
- **Indirect**，即当 GPU 之间没有直接链路时，必须通过其 CPU 的 DRAM 中转数据，从而带来额外的内存拷贝开销。

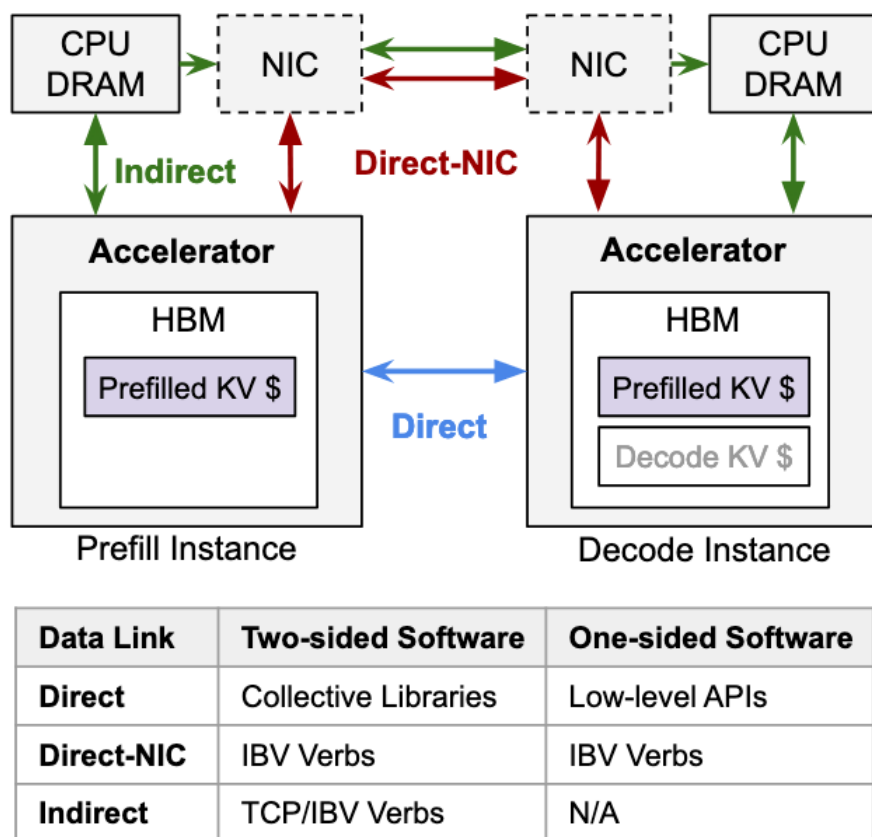


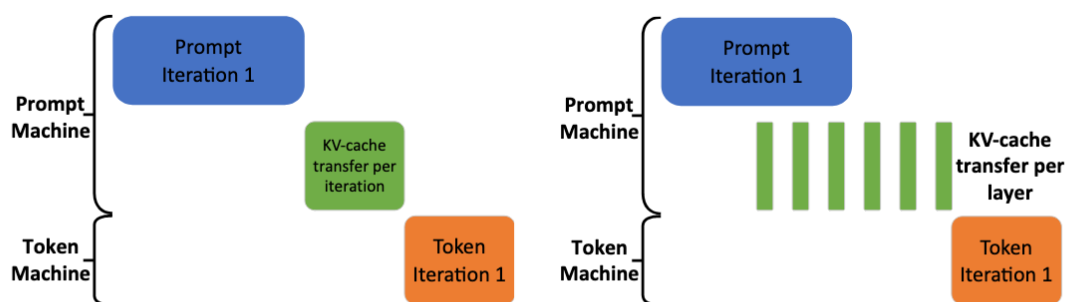
Figure 9: **Data Links and Network Stacks.** The *Direct* link provides bandwidth in the hundreds of GBs (e.g, NVLink 900GBps). The *Direct-NIC* and *Indirect* link offer bandwidth in the hundreds of Gbs (e.g, ConnectX-6 200Gbps). The 1-sided stack means the sender accelerator can transmit data to the receiver accelerator without involving the receiver’s CPU.

图片来源: [Inference without Interference: Disaggregate LLM Inference for Mixed Downstream Workloads](#)

5.4 KV Cache 传输粒度

KV cache 传输粒度可以分为 3 类:

- **请求级:** 等到 prefill 阶段完成后, 将 KV cache 一次性传输。这种方式的好处是能够减少网络传输次数, 因为每次传输的数据量更大, 从而降低了通信开销。然而当 KV cache 大小较大时, 会影响 TTFT 的延迟。
- **层级:** Splitwise 通过在 prefill 阶段的计算与 KV cache 传输之间实现重叠来优化性能。每一层计算完成后, 都会异步传输该层的 KV cache, 同时继续执行下一层的计算, 从而降低传输开销。层级传输还能带来额外优势, 例如更早启动 decode 阶段, 以及更早释放 prefill 端的内存。层级 KV cache 传输与下一层的 prefill 计算并行进行, 这需要逐层的细粒度同步以确保正确性, 因此可能会带来性能干扰并增加 TTFT, 尤其是在小 prompt 的场景下。不过对于小 prompt 来说, KV cache 的总体规模很小, 不需要层级传输来隐藏延迟。由于在计算开始时批次中的 token 数已经是已知的, Splitwise 会选择最合适的 KV cache 传输方式: 小 prompt 使用序列化传输, 而大 prompt 使用层级传输。



(a) Serialized KV-cache transfer. (b) Optimized KV-cache transfer per-layer during prompt phase.

Fig. 11: Optimizing KV-cache transfer in Splitwise.

图片来源: [Splitwise: Efficient Generative LLM Inference Using Phase Splitting](#)

- **块级**: TetriInfer 在 PD 分离的基础上, 还会将输入的 prompt 划分为固定大小的 chunk, 以便让 GPU 始终运行在接近计算饱和的状态。因此, TetriInfer 论文中也提出了基于块级的 KV cache 传输方案。

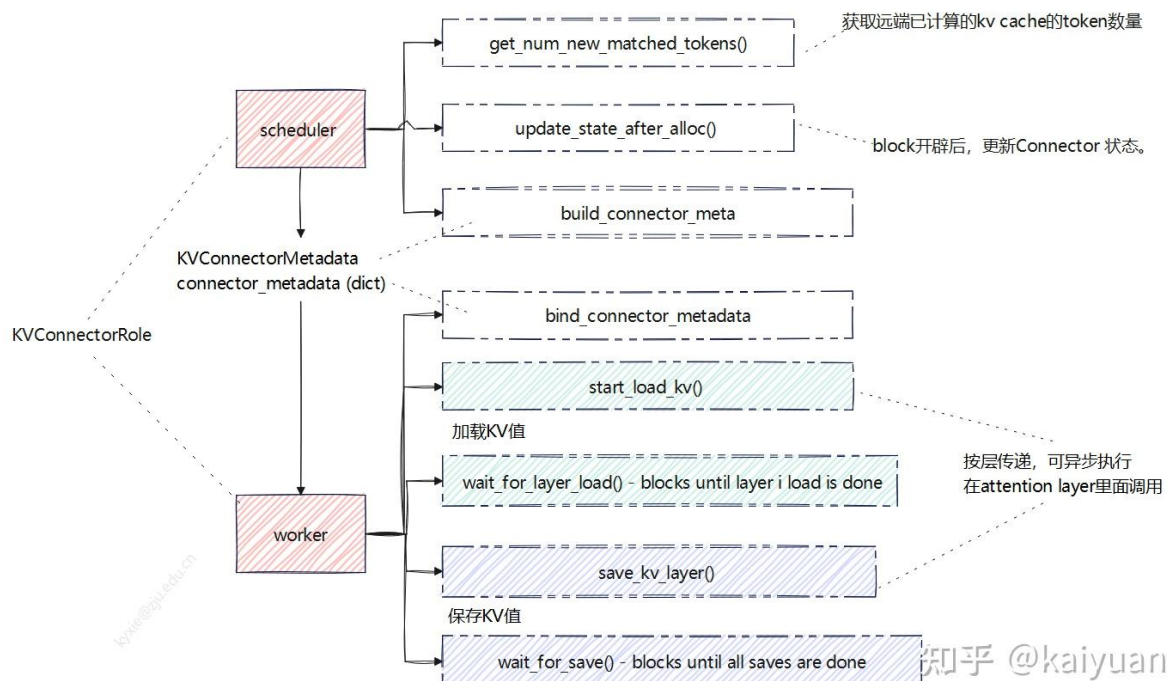
6vLLM 的 PD 分离

vLLM 提供了 **KV Connector** 作为管理实例间 KV cache 交换的抽象层, 它提供统一接口来实现 KV cache 的保存、加载与传输, 使不同的 vLLM 实例 (如 prefill 与 decode 实例) 能够高效共享计算结果。通过实现这一接口, 各类 connector (例如通过文件系统的 SharedStorageConnector、通过网络的 NixlConnector 等) 提供了灵活的 KV cache 传输方案, 从而支持 PD 分离等高级功能。

`KVConnectorBase_v1` 是所有 connector 的基类。它是一个抽象基类, 定义了以下 API:

- scheduler 侧方法:
 - `build_connector_meta`: 构建元数据, scheduler 告诉 worker 需要保持/加载哪些 KV cache。
 - `get_num_new_matched_tokens`: 获取远端已计算的 KV cache 的 token 数量。
 - `update_state_after_alloc`: block 开辟后, 更新 connector 的状态。
- worker 侧方法:
 - **`start_load_kv`**: 从 connector buffer 加载 KV cache, 消费端调用。
 - `wait_for_layer_load`: 阻塞直到指定层加载结束, 消费端调用;
 - **`save_kv_layer`**: 将 vLLM 的 KV buffer 中某一层的 KV cache 保存到 connector buffer 中, 生产端调用。
 - `wait_for_save`: 阻塞直到所有保存操作完成, 生产端调用。

vLLM v1 中 connector 有两个执行角色 (Role): `scheduler_connector` 和 `worker_connector`, 分别在 scheduler 线程和 worker 线程中执行。**scheduler 负责指挥 worker 进行 KV cache 的传递, 两者之间的信息桥梁是元数据 (KVConnectorMetadata), worker 通过 metadata 知道哪些 KV 值需要从远端加载。**



当前 vLLM 支持 5 种类型的 connector, 分别是:

- **SharedStorageConnector**: SharedStorageConnector 是 vLLM 中最简单的 KV Connector 实现, 它通过共享文件系统 (如本地磁盘或 NFS) 在 prefill 和 decode 实例之间传递 KV cache, 使用 MD5 哈希生成唯一文件名来存储和检索每个请求的 KV cache。prefill 实例将每层的 KV cache 序列化为 SafeTensors 格式保存到指定路径, decode 实例根据相同的 token_ids 计算哈希值找到对应文件并加载, 整个过程没有显式的网络传输, 完全依赖文件系统的读写操作。
- **P2pNcclConnector**: P2pNcclConnector 是基于 NCCL (NVIDIA Collective Communications Library) 实现的高性能 KV Connector, 它通过 NCCL 的 send/recv 原语实现 KV cache 在不同 GPU 之间的点对点传输, 避免了文件系统的开销。
- **NixlConnector**: NixlConnector 使用 NIXL (NVIDIA Inference Xfer Library) 库来加速 GPU 之间以及异构内存与存储之间的 KV cache 传输。
- **LMCacheConnectorV1**: 通过与 LMCache 集成实现 KV cache 的外部存储和检索, 支持多种 [存储后端](#) (如 CPU 内存、本地文件系统、Redis、InfiniStore 等)。LMCache 通过重用缓存的 KV cache 来减少推理时间, 消除冗余计算, 适用于跨请求或跨会话的 KV cache 共享场景。
- **MultiConnector**: 允许同时使用多个 KV connector 来实现 KV cache 的传输, 它的核心逻辑是从第一个能提供可用 token 的 connector 加载 KV cache, 但会向所有 connector 保存数据。MultiConnector 适用于需要同时向多个存储后端保存 KV cache 的场景, 比如同时保存到本地存储和远程存储, 提供数据冗余和可靠性保障。

```
--kv-transfer-config '{
  "kv_connector": "MultiConnector",
  "kv_connector_extra_config": {
    "connectors": [
      {
        "kv_connector": "NixlConnector",
        "kv_role": "kv_both"
      },
      {
        "kv_connector": "SharedStorageConnector",
        "kv_connector_extra_config": {
          "shared_storage_path": "local_storage"
        }
      }
    ]
  }
}
```

```

    "kv_role": "kv_both"
  }
]
},
"kv_role": "kv_both"
}'

```

以上几个 connector 的运行实例代码可以在这里找到: https://docs.vllm.ai/en/latest/features/disagg_prefill.html#usage-example

7 PD 分离工业界项目

7.1 Mooncake

[Mooncake](#) 是 Moonshot AI 提供的领先大模型服务 Kimi 的推理平台。Mooncake 是 PD 分离应用比较早也是规模比较大的成功例子:

- Mooncake 采用了以 KV cache 为核心的解耦架构, 将 prefill 集群与 decode 集群分离。
- 同时, 它还利用 GPU 集群中未被充分利用的 CPU、DRAM 和 SSD 资源, 实现了解耦式的 KV cache 缓存。
- Mooncake 的核心是一个以 KV cache 为中心的调度器, 它在最大化整体有效吞吐量和满足延迟相关的 SLO 之间进行平衡。
- 与传统假设所有请求都会被处理的研究不同, Mooncake 需要应对高负载场景带来的挑战。为此, Mooncake 提出了一种基于预测的早期拒绝策略。

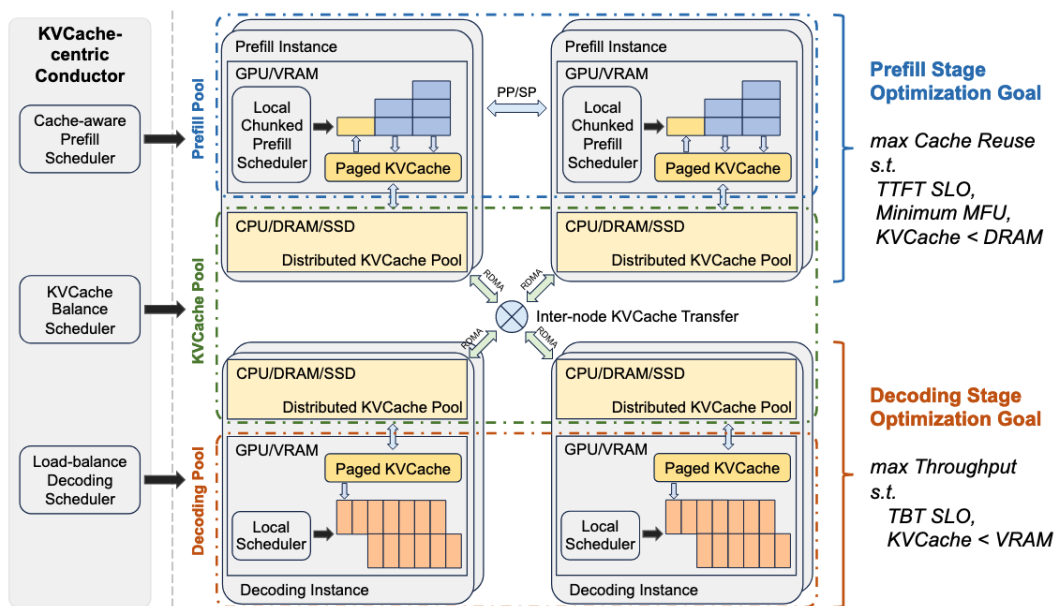


Figure 1: Mooncake Architecture.

图片来源: [Mooncake: A KVCACHE-centric Disaggregated Architecture for LLM Serving](#)

实验结果表明, Mooncake 在长上下文场景下表现尤为突出: 在某些模拟场景中, 吞吐量相比基线方法最高可提升 525%, 同时仍能满足 SLO。在真实负载下, Mooncake 的创新架构使 Kimi 能够多处理 75% 的请求。

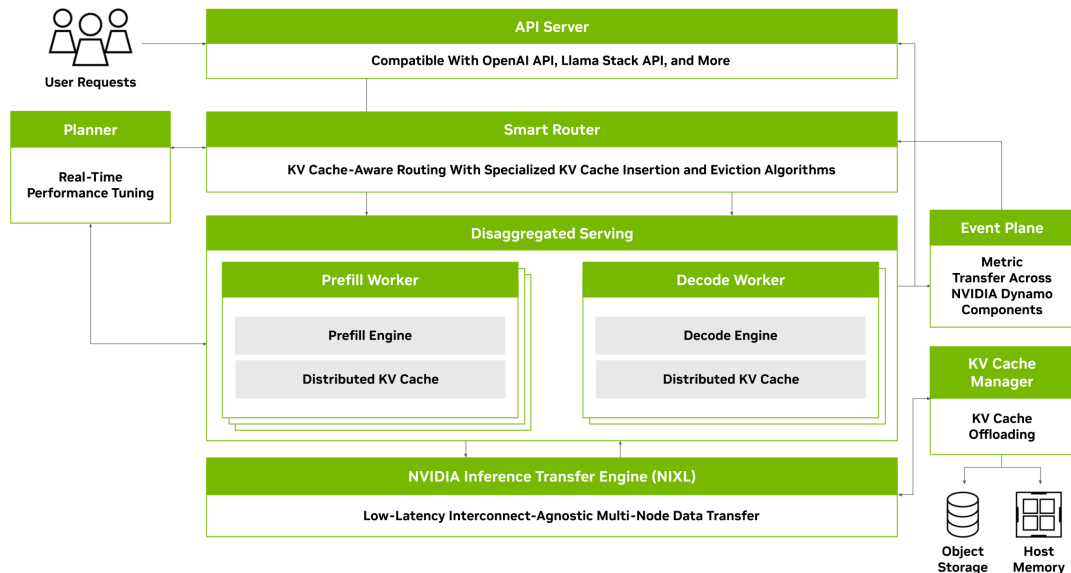
使用 Mooncake 运行 PD 分离服务请参考文档: [vLLM V1 Disaggregated Serving with Mooncake Store and LMCACHE](#)

7.2 Dynamo

[NVIDIA Dynamo](#) 是一个开源的模块化推理框架，用于在分布式环境上实现生成式 AI 模型的服务化部署。Dynamo 通过动态资源调度、智能路由、内存优化与高速数据传输，无缝扩展大型 GPU 集群之间的推理工作负载。

Dynamo 采用推理引擎无关的设计（支持 TensorRT-LLM、vLLM、SGLang 等），包括以下 4 个核心组件：

- **NVIDIA Dynamo Planner**：一个智能规划和调度引擎，用于监控分布式推理中的容量与延迟，并在 prefill 与 decode 阶段之间灵活分配 GPU 资源，以最大化吞吐量和效率。Planner 会持续跟踪关键的 GPU 容量指标，并结合应用的 SLO（如 TTFT 和 ITL），智能决策是否采用分离式推理，或是否需要为 prefill/decode 阶段动态增加更多 GPU。
- **NVIDIA Dynamo Smart Router**：KV cache 感知的路由引擎，可在分布式推理环境中将请求转发到最佳的节点，从而最大限度减少 KV cache 的重复计算开销。
- **NVIDIA Dynamo Distributed KV Cache Manager**：通过将较旧或低频访问的 KV cache 卸载到更低成本的存储（如 CPU 内存、本地存储或对象存储等），大幅降低 GPU 内存占用。借助这种分层管理，开发者既能保留大规模 KV cache 重用的优势，又能释放宝贵的 GPU 资源，从而有效降低推理计算成本。
- **NVIDIA Inference Transfer Library (NIXL)**：高效的推理数据传输库，可加速 GPU 之间以及异构内存与存储之间的 KV cache 传输。通过减少同步开销和智能批处理，NIXL 显著降低了分布式推理中的通信延迟，使得在 prefill/decode 分离部署时，prefill 节点也能在毫秒级将大批量的 KV cache 传输至 decode 节点，从而避免跨节点数据交换成为性能瓶颈。

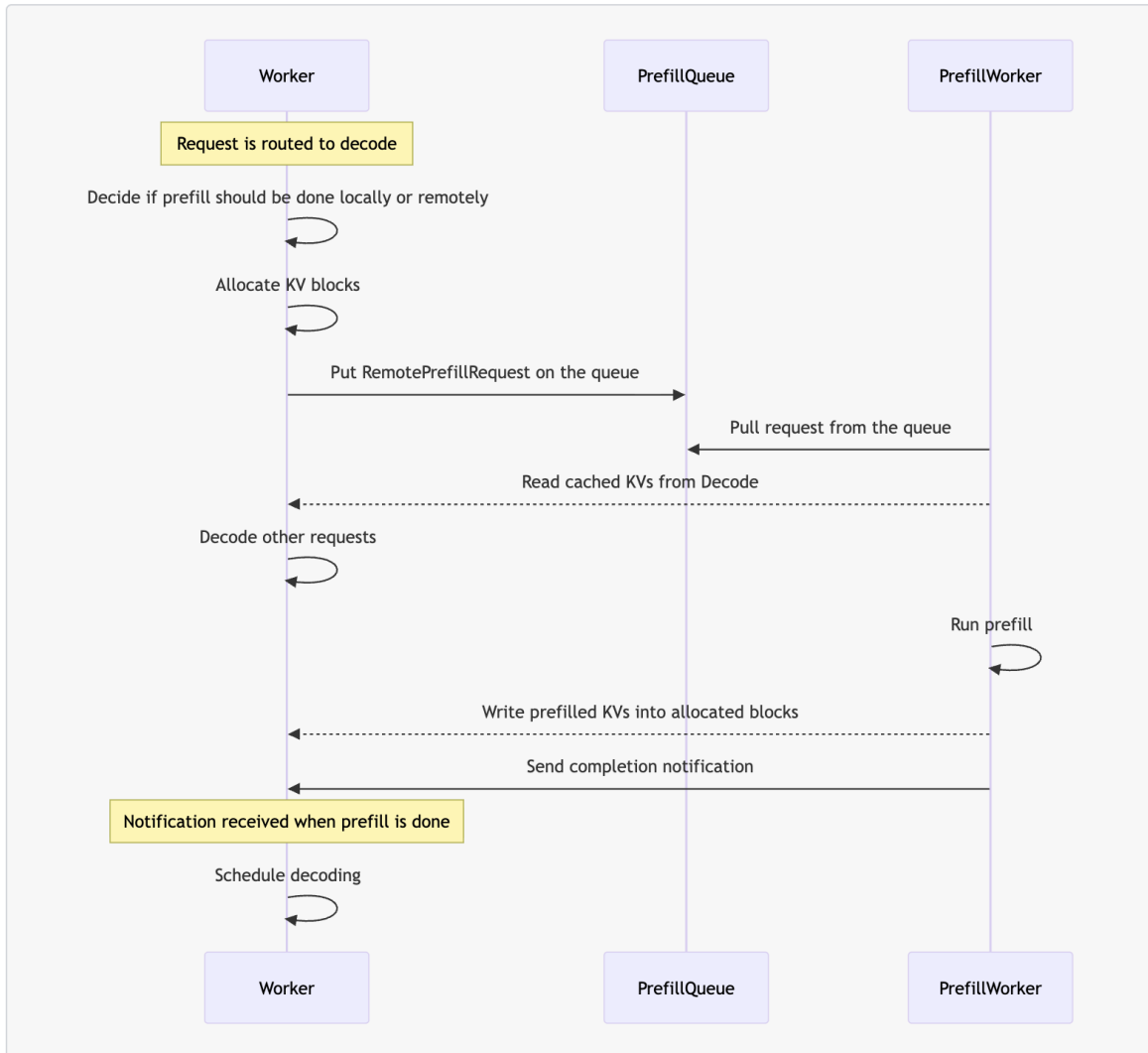


在 Dynamo 的 PD 分离架构中，有 4 个核心组件：

- **(decode) worker**：执行 prefill 和 decode 请求。
- **prefill worker**：只执行 prefill 请求。
- **disaggregated router**：决定 prefill 阶段是在本地还是远程执行。
- **prefill queue**：缓存并负载均衡远程 prefill 请求。

当 worker 收到请求时，首先会通过 disaggregated router 判断 prefill 应该在本地还是远程完成，并分配相应的 KV block。

如果选择远程 prefill，请求会被推送到 prefill queue。随后，prefill worker 从队列中取出请求，读取 worker 中 prefix cache 命中的 KV block，执行 prefill 计算，并将生成的 KV block 回写给 worker。最后，worker 会继续完成剩余的 decode 阶段。



Dynamo 提供了 Operator 方便我们在 Kubernetes 环境中以声明式的方式定义 PD 分离服务。只需在 `DynamoGraphDeployment` 配置中声明 Frontend、VllmDecodeWorker 和 VllmPrefillWorker 三个组件即可。`dynamoNamespace` 是 Dynamo 分布式运行时的逻辑隔离单元，而非 Kubernetes 的 namespace；同一 `dynamoNamespace` 内的组件可以相互发现并进行通信。

```
apiVersion: nvidia.com/v1alpha1
kind: DynamoGraphDeployment
metadata:
  name: vllm-disagg
spec:
  services:
    Frontend:
      dynamoNamespace: vllm-disagg
      componentType: frontend
      replicas: 1
      extraPodSpec:
        mainContainer:
          image: nvcr.io/nvidia/ai-dynamo/vllm-runtime:0.4.1
    vllmDecodeWorker:
```

```

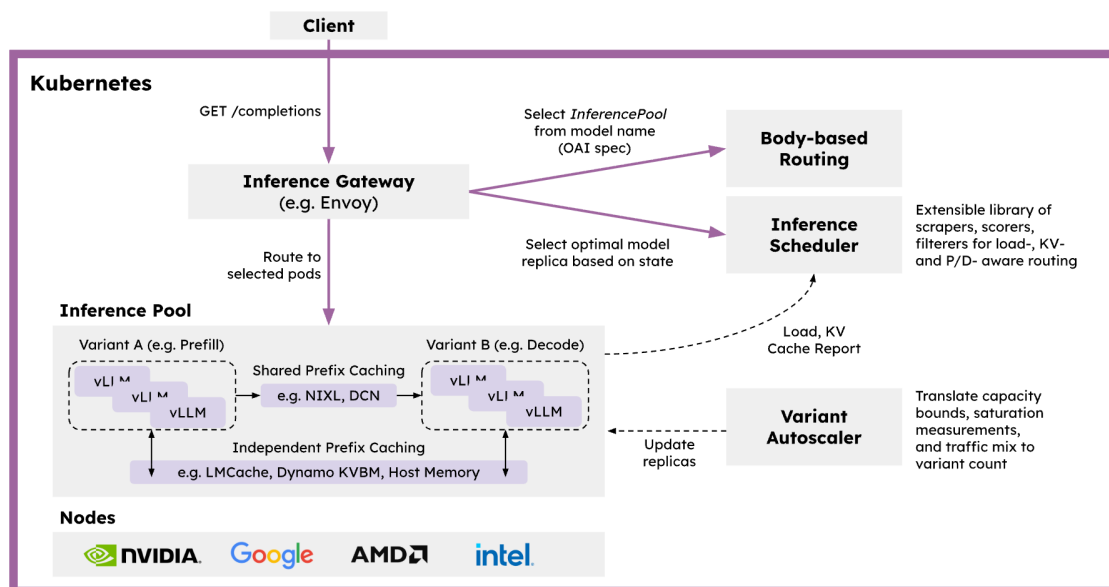
dynamoNamespace: vllm-disagg
componentType: worker
replicas: 1
resources:
  limits:
    gpu: "1"
extraPodSpec:
  mainContainer:
    image: nvcr.io/nvidia/ai-dynamo/vllm-runtime:0.4.1
    workingDir: /workspace/components/backends/vllm
    command:
      - /bin/sh
      - -c
    args:
      - "python3 -m dynamo.vllm --model Qwen/Qwen3-0.6B"
vllmPrefillWorker:
dynamoNamespace: vllm-disagg
componentType: worker
replicas: 1
resources:
  limits:
    gpu: "1"
extraPodSpec:
  mainContainer:
    image: nvcr.io/nvidia/ai-dynamo/vllm-runtime:0.4.1
    workingDir: /workspace/components/backends/vllm
    command:
      - /bin/sh
      - -c
    args:
      - "python3 -m dynamo.vllm --model Qwen/Qwen3-0.6B --is-prefill-
worker"

```

Dynamo 详细的部署教程可以参考博客：<https://cr7258.github.io/blogs/original/2025/20-dynamo#3-%E8%BF%90%E8%A1%8C-dynamo>

7.3 llm-d

[llm-d](#) 是一个 Kubernetes 原生的分布式推理服务栈，为团队提供一条清晰高效的路径，以最快的落地速度在大规模环境中部署并管理推理服务。llm-d 通过集成业界标准的开源技术来加速分布式推理：使用 vLLM 作为模型服务与引擎，Inference Gateway 作为请求调度器与负载均衡器，Kubernetes 作为基础设施编排与工作负载控制平面。



llm-d 提供以下核心功能：

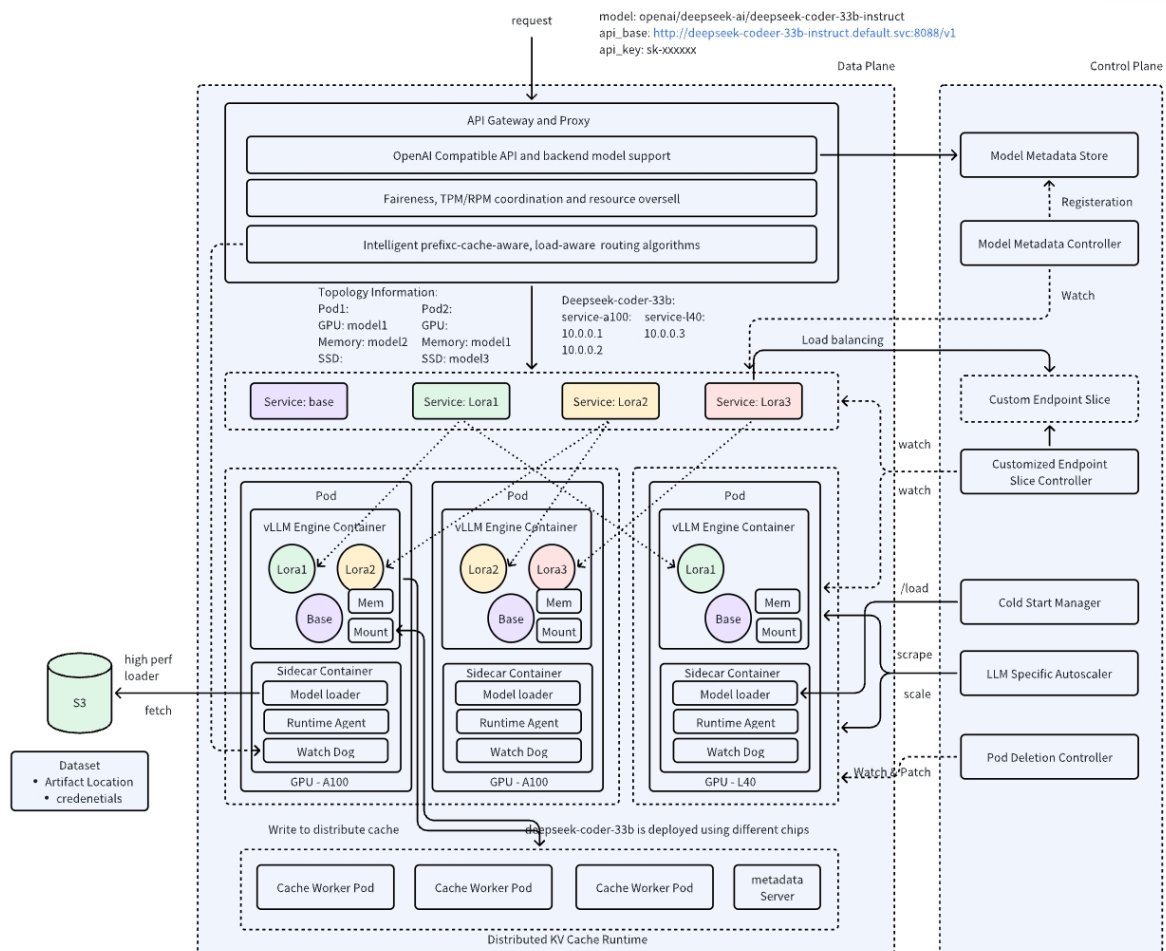
- 基于 vLLM 优化的推理调度器：llm-d 基于 IGW 的 Endpoint Picker Protocol (EPP) 实现可定制化的“智能”负载均衡，专门针对 vLLM 进行优化。调度器结合运行时遥测数据，利用过滤和打分算法，在 P/D 分离、KV cache、SLA 与负载感知的基础上做出调度决策，同时支持团队自定义策略。
- PD 分离：llm-d 利用 vLLM 的分离式推理能力，将 prefill 和 decode 拆分到独立实例运行，并通过高性能传输库（如 NIXL）进行通信。
- 分布式 KV cache：llm-d 使用 vLLM 的 KV connector 构建可插拔的 KV cache 层级体系，支持将 KV cache 卸载到主机、远程存储或 LMCACHE 等系统。

使用 llm-d 运行 PD 分离服务请参考：<https://github.com/llm-d/llm-d/tree/main/guides/pd-disaggregation>

7.4 AIBrix

AIBrix 是字节跳动开源的云原生分布式推理框架，专为大规模 LLM 部署设计。

- AIBrix 支持 LoRA 管理、前缀感知和负载感知的智能路由，并通过分布式 KV cache 实现跨节点的高效 token 复用。
- 在系统编排层面，AIBrix 结合 Kubernetes 与 Ray 的混合调度，既能满足大规模集群管理的需求，又能灵活执行细粒度任务。
- AIBrix 基于 SLO 的 GPU 优化器与诊断工具进一步提升了资源利用率和系统可靠性。



使用 AIBrix 运行 PD 分离服务请参考：<https://github.com/vllm-project/aibrix/tree/main/samples/disaggregation/vllm>

8 Chunked-Prefills VS PD 分离

chunked-prefills 方案的核心思想是：将长序列的预填充请求拆分为几乎相等大小的小块，然后构建了由 prefill 小块和 decode 组成的混合 batch。或者说，chunked-prefills 策略通过将不同长度的 prompts 拆分成长度一致的 chunks 来进行 prefill，以避免长 prompt 阻塞其他请求，同时利用这些 chunks 的间隙进行 decode 的插入/捎带 (piggyback) 操作，从而减少延迟并提高整体的吞吐。

decode 阶段的开销不仅来自从 GPU 内存中获取 KV cache，还包括提取模型参数。而通过这种 piggyback 方法，decode 阶段能够重用 prefill 时已提取的模型参数，几乎将 decode 阶段从一个以内存为主的操作转变为一个计算为主的操作。因此，这样构建的混合批次具有近乎均匀的计算需求（而且增加了计算密集性），使我们能够创建平衡的微批处理调度，缓解了迭代之间的不平衡，导致 GPU 的管道气泡最小化，提高了 GPU 的利用率。也最小化了计算新 prefill 对正在进行的 decode 的 TBT 的影响，从而实现了高吞吐量和低 TBT 延迟。

chunked-prefills 有两个明显的好处：

- 所有节点被平等对待，使调度更简单。
- 将 chunked prefill 内联到 decode 批处理中可以提高 decode 批次的计算强度，从而带来更好的 MFU (Model FLOPs Utilization, 指的是模型实际使用的计算量占 GPU 理论峰值算力的比例，用来衡量算力利用效率)。

然而，chunked-prefills 也有一些缺点：

- chunked-prefills 会增加 prefill 的计算开销，如果 chunk 大小明显低于 GPU 饱和点，会延长 prefill 的执行时间。

- prefill 阶段仍难以完全最大化 MFU，因为在 chunk-prefill 中，profiling 只会估算特定设备上一个 batch 的最大 tokens 配额，这个配额同时包含 prefill 和 decode，而不是分别针对两者优化。
- chunked-prefills 也会显著增加 prefill 阶段的内存访问量，每个 chunk 的 Attention 操作都需重复读取此前的 KV cache，增加内存访问负担。而且长序列可能会持久地占据着 KV cache 的存储空间以及 GPU 的计算资源。
- 在 TPOT 方面，将 prefill 与 decode 合并批处理实际上会降低所有 decode 任务的平均速度。

总之，chunked-prefills 可能有助于最大化整体吞吐量，但由于动态分割无法完全解耦 prefill 和 decode 操作，会导致资源争用以及 TTFT 与 TPOT 之间的妥协。当应用程序无法在 TTFT 和 TPOT 之间进行权衡，而是要同时遵守这两者时，PD 分离就成为更好的选择。

9 PD 分离相关论文

- [DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving](#)
- [Splitwise: Efficient Generative LLM Inference Using Phase Splitting](#)
- [TetriInfer: Inference without Interference: Disaggregate LLM Inference for Mixed Downstream Workloads](#)
- [MemServe: Context Caching for Disaggregated LLM Serving with Elastic Memory Pool](#)
- [Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving](#)

10 总结

11 参考资料

- Lecture 58: Disaggregated LLM Inference: <https://www.youtube.com/watch?v=tIPDwUepXcA>
- Throughput is Not All You Need: Maximizing Goodput in LLM Serving using Prefill-Decode Disaggregation: <https://hao-ai-lab.github.io/blogs/distserve/>
- Mooncake阅读笔记：深入学习以Cache为中心的调度思想，谱写LLM服务降本增效新篇章: <http://zhuanlan.zhihu.com/p/706097807>
- 探秘Transformer系列之（26）--- KV Cache优化 之 PD分离or合并: <https://www.cnblogs.com/rossiXYZ/p/18815541>
- 大模型推理分离架构五虎上将: <https://zhuanlan.zhihu.com/p/706218732>
- LLM关于PD分离的最新实测: <https://zhuanlan.zhihu.com/p/1919794916504114120>
- State of the Model Serving Communities - August 2025: <https://inferenceops.substack.com/p/state-of-the-model-serving-communities>
- 图解大模型训练系列：序列并行1，Megatron SP: <https://zhuanlan.zhihu.com/p/4083427292>
- 序列并行做大模型训练，你需要知道的六件事: <https://zhuanlan.zhihu.com/p/698031151>
- vLLM PD分离KV cache传递机制详解与演进分析: <https://zhuanlan.zhihu.com/p/1906741007606878764>
- vLLM PD分离方案浅析: <https://zhuanlan.zhihu.com/p/1889243870430201414>
- P/D Disaggregation of vLLM and Integration with Mooncake: https://docs.google.com/document/d/1Ab6TMW1E2CdHJyCrpJnLhgmE2b_6leH5MVP9k72sjw/edit?tab=t.0#heading=h.611v2r4aqubz
- 0.5x提升:PD分离KV cache传输的实践经验: <https://zhuanlan.zhihu.com/p/1946608360259577576>

- 分布式推理优化思路: <http://zhuanlan.zhihu.com/p/193755622371946860>
- 图解大模型计算加速系列: 分离式推理架构2, 模糊分离与合并边界的chunked-prefills: <https://zhuanlan.zhihu.com/p/710165390>
- 图解大模型计算加速系列: 分离式推理架构1, 从DistServe谈起: <https://zhuanlan.zhihu.com/p/706761664>
- LLM推理优化 - Prefill-Decode分离式推理架构: <https://zhuanlan.zhihu.com/p/9433793184>
- Shaping NIXL-based PD Disaggregation in vLLM V1: <https://blog.lmcache.ai/2025-04-11-lmcache-vllmv1-nixl/>
- vLLM P2P NCCL Connector: https://docs.vllm.ai/en/latest/design/p2p_nccl_connector.html
- vLLM Disaggregated Prefilling (experimental): https://docs.vllm.ai/en/latest/features/disagg_prefill.html
- LMCACHE Example: Disaggregated prefill: https://docs.lmcache.ai/getting_started/quickstart/disaggregated_prefill.html
- Bringing State-Of-The-Art PD Speed to vLLM v1 with LMCACHE: <https://blog.lmcache.ai/2025-04-29-pdbench/>
- Demystify vLLM V1 KVconnector SharedStorageConnector: <https://blog.diabloneo.com/demystify-vllm-v1-kvconnector-sharedstorageconnector-05a487627036>
- vLLM源码之分离式架构: <https://zhuanlan.zhihu.com/p/1933647687>
- vLLM v1 PD分离设计: <https://zhuanlan.zhihu.com/p/1894425784107632241>
- Inside vLLM: Anatomy of a High-Throughput LLM Inference System: <https://blog.vllm.ai/2025/09/05/anatomy-of-vllm.html>
- [P/D][V1] KV Connector API V1: <https://github.com/vllm-project/vllm/pull/15960>
- vLLM PD Disaggregation discussion: <https://docs.google.com/document/d/1uPGdbEXksKXeN4Q9nUm9hzotqEjQhYmnpAhidLuAsjk/edit?tab=t.0#heading=h.qhtgj3vmvwn>
- llm-d: Kubernetes-native Distributed Inference at Scale: <https://github.com/llm-d/llm-d/blob/dev/docs/proposals/llm-d.md>